Operating Systems

Model Answers: 20-Mark Debugging Questions (Q1 to Q5)

Q1. Deadlock Debugging (Code Trace)

(20 Marks | BL4)

1. Code under analysis

```
P1: lock(R1); wait(100ms); lock(R2); CS; unlock(R2); unlock(R1)
P2: lock(R2); wait(100ms); lock(R1); CS; unlock(R1); unlock(R2)
```

2. Execution trace showing the deadlock

Time	P1	P2
t = 0	lock(R1) is granted	lock(R2) is granted
t = 0 to 100 ms	wait(100 ms) while holding R1	wait(100 ms) while holding R2
t = 100 ms	lock(R2) blocks because P2 owns R2	lock(R1) blocks because P1 owns R1
t > 100 ms	Blocked on P2 indefinitely	Blocked on P1 indefinitely

3. The exact condition behind the deadlock

The system is stuck in a **circular wait**. P1 and P2 take the same two locks in **opposite orders** (P1 takes R1 then R2, while P2 takes R2 then R1), so each ends up waiting for a lock the other one holds. The wait-for relationship is:

```
P1 holds R1  -->  R1 is requested by P2
P2 holds R2  -->  R2 is requested by P1
Wait-for graph: P1 -> P2 -> P1  (a cycle)
```

At this point all four **Coffman conditions** hold together:

Condition	How it appears in this code
Mutual exclusion	A lock gives one process exclusive use, so R1 and R2 can each be owned by only one process at a time.
Hold and wait	P1 keeps R1 while asking for R2; P2 keeps R2 while asking for R1.
No pre-emption	A lock cannot be taken away by force; it is freed only when its owner calls unlock().
Circular wait	P1 waits on P2 and P2 waits on P1, which closes the cycle.

Role of wait(100 ms): the delay is not the cause of the deadlock. It only lengthens the period in which each process holds its first lock, so the bad interleaving occurs on nearly every run. Without the delay the flaw would remain, but it would show up only on an unlucky schedule, which makes it a timing-dependent bug that is hard to reproduce.

4. Optimisation that removes the deadlock

Break the circular wait with a global resource ordering, the standard deadlock-prevention technique. Give every resource a fixed number (R1 = 1, R2 = 2) and make each process request locks strictly in ascending order. The critical section and the surrounding logic are untouched; only the order of acquisition is made the same for everyone. This is normally done once in a lock-acquisition helper, so the process bodies need no change:

```
acquire_both(a, b):          // helper enforces the global order
    first  = lower_id(a, b)
    second = higher_id(a, b)
    lock(first); lock(second)

P1: acquire_both(R1, R2); CS; release both
P2: acquire_both(R2, R1); CS; release both   // internally locks R1 then R2
```

Why this works: both processes now contend for R1 first. The winner continues to R2, finishes, and releases both; the loser waits at R1 while holding nothing. A cycle cannot form, because a process that owns a higher-numbered resource never waits for a lower-numbered one.

Other acceptable approaches (worth listing for full marks):

- **Atomic acquisition (breaks hold-and-wait):** take R1 and R2 together as one all-or-nothing step; if both are not available, hold neither.
- **trylock with timeout and back-off (breaks no-pre-emption):** if lock(R2) does not succeed in time, release R1, wait a random interval, and try again.
- **Deadlock avoidance:** the OS approves a request only when the resulting state is still safe (Banker's algorithm).
- **Detection and recovery:** periodically build a resource-allocation graph, find the cycle, then abort or roll back one of the processes.

Conclusion: the root cause is inconsistent lock ordering, which produces a circular wait. A single global lock order removes the cycle, costs almost nothing, and does not reduce concurrency.

Q2. CPU Utilisation Problem

(20 Marks | BL4)

1. What the log shows

The CPU is only 30% busy even though processes are present, and the scheduler log keeps repeating ready, waiting, ready, waiting. In a healthy system the usual cycle is ready, running, waiting (blocked), ready. Here the processes hardly get to the running state, and when they do, they use the CPU for a very short burst and then block again.

```
Normal  : READY -> RUNNING -> WAITING -> READY -> RUNNING ...
Observed: READY -> (tiny run) -> WAITING -> READY -> (tiny run) -> WAITING ...
```

2. Debugging with process-management concepts

Utilisation is low because, at any moment, most processes sit in the waiting (blocked) queue while the ready queue is empty or emptied almost immediately. A process is "ready" only for a moment before it blocks again. The likely causes are:

1. **I/O-bound workload with very short CPU bursts:** each process asks for I/O almost at once, so the CPU idles while devices respond. Utilisation is modelled as 1 - p^n, where p is the fraction of time a process spends waiting and n is the degree of multiprogramming. For p =

0.8, n = 4 gives 1 - 0.8^4 = 59%, while n = 2 gives only 36%. A small n therefore means low utilisation.

2. **Low degree of multiprogramming or a poor job mix:** the long-term scheduler admits too few processes, or all of them are I/O-bound, so no CPU-bound process is around to use the idle gaps (the convoy effect).
3. **Thrashing (excessive paging):** when memory is over-committed, processes keep waiting for page-fault service and cycle through ready, waiting, ready, waiting. This is the classic signature of low CPU use together with heavy disk activity.
4. **Blocking I/O and lock contention:** processes stall on slow I/O or on a contended lock, with no overlap between computation and I/O.

Diagnosis steps: inspect vmstat, iostat and top for the I/O-wait percentage and the page-fault rate. High wa or heavy paging points to I/O or memory trouble; a run-queue length close to zero means the degree of multiprogramming is too low; long lock-wait times indicate contention.

3. System-level optimisations

Optimisation	Effect on CPU utilisation
Balance the job mix (long-term scheduler)	Admit both CPU-bound and I/O-bound processes so the CPU always has work while others wait for I/O.
Raise the degree of multiprogramming (with care)	More resident processes make it more likely that at least one is ready. Limit it with the working-set model so it does not cause thrashing.
Overlap I/O with computation	Use DMA, asynchronous or non-blocking I/O, buffering, caching, spooling and read-ahead so processes block less often and for less time.
Use a Multilevel Feedback Queue scheduler	I/O-bound and interactive processes get high priority, so they reach the CPU quickly when their I/O completes, while CPU-bound jobs fill the idle slots.
Fix thrashing (if page faults are the cause)	Apply the working-set or page-fault-frequency policy to give each process enough frames, and swap out some processes.
Reduce lock contention	Use finer-grained locks or lock-free structures so processes do not block one another.

Conclusion: the processes spend most of their time waiting, which leaves the CPU idle. The best system-level remedy is to balance the job mix and increase the degree of multiprogramming within memory limits, and at the same time overlap I/O with computation (asynchronous I/O, buffering, MLFQ). Utilisation then rises well above 30%.

Q3. Memory Bottleneck Debugging

(20 Marks | BL4)

1. Analysis of the memory map

Region	Size
Code (text, shared by all threads)	5 MB

Region	Size
Heap (shared by all threads)	150 MB
Stacks: 25 threads x 1 MB	25 MB
Total virtual footprint	5 + 150 + 25 = 180 MB
Physical RAM	2 GB (2048 MB)

The program needs about 180 MB, roughly 9% of the 2 GB of RAM, so the page faults are **not due to a lack of physical memory** (unless other processes are using up the RAM). The trouble lies in how the memory is used and accessed.

2. The memory problem

The central problem is **poor locality of reference over a large shared heap, together with demand-paging and stack-growth faults across 25 threads**:

- **Heap access pattern:** 25 threads operate on a 150 MB shared heap. If their accesses are scattered, the working set is not compact, which produces many page faults and TLB misses. The heap may also be fragmented and serialised behind a single allocator.
- **Demand paging (cold faults):** heap and stack pages are loaded only when first touched, so start-up and fresh allocations generate many minor and major faults.
- **Stack pressure:** 1 MB per thread is small; deep recursion or large local arrays trigger stack-growth faults or guard-page hits. Twenty-five stacks also compete for cache and TLB entries.
- **External pressure:** if other processes or the page cache consume RAM, this program's pages get swapped out and must be faulted back in.

Cost of page faults: Effective Access Time EAT = (1 - p) x memory access time + p x page-fault service time. Taking memory access = 100 ns and fault service = 8 ms, even a fault rate of p = 0.001 gives EAT = 0.999 x 100 + 0.001 x 8,000,000, which is about 8,100 ns. That is roughly 80 times slower than a normal access, so a very small fault rate ruins performance.

3. Two optimisations

Optimisation 1: Improve locality and reduce heap contention.

- Give each thread its own allocator or arena so that it works inside a compact region of the heap; this also removes allocator lock contention and false sharing.
- Reshape data and loops (array-of-structs versus struct-of-arrays, blocking or tiling) so threads walk memory sequentially and the working set fits in RAM and cache.
- Result: fewer distinct pages are touched, so there are fewer page faults and TLB misses.

Optimisation 2: Pre-fault, pin and enlarge pages for hot data.

- Allocate and pre-fault memory at start-up (for example mmap with MAP_POPULATE, or touch each page once), and use a memory pool in place of repeated malloc/free.
- Pin the hot working set in RAM (mlock) and lower swappiness so it is not swapped out.
- Use 2 MB huge pages for the heap to shrink the page-table and reduce TLB misses, and prefetch with madvise(WILLNEED).
- Result: the fault cost is paid once at the beginning rather than during execution.

Further tuning: size each thread stack to its real need and avoid deep recursion, match the thread count to the number of CPU cores instead of using 25 threads blindly, and use a clock (LRU-approximation) page-replacement policy with enough frames per process.

Conclusion: the slowdown comes from poor locality and demand-paging overhead, not from a shortage of RAM. Thread-local allocation with better data locality, combined with pre-faulting, pinning and huge pages, cuts page faults sharply.

Q4. Starvation Debugging

(20 Marks | BL4)

1. Scheduler configuration

Queue	Type / Algorithm	Priority
Q1	Interactive, Round Robin (quantum 5 ms)	Highest
Q2	System, Round Robin (quantum 10 ms)	Medium
Q3	Batch, FCFS	Lowest (P5 is here)

2. Root cause of the starvation

A basic multilevel queue applies **fixed-priority scheduling between queues**: a lower queue gets the CPU only when every higher queue is empty. Q3 therefore runs only when Q1 and Q2 are both empty. On a busy system, interactive jobs (Q1) and system jobs (Q2) keep arriving, so those queues are almost never empty. P5 is left waiting **without limit**, which is **starvation (indefinite blocking)**.

```
CPU timeline: Q1 Q1 Q2 Q1 Q1 Q2 Q1 Q2 Q1 ...   (Q3 never gets a turn)
P5 (in Q3) keeps waiting as long as any Q1 or Q2 process is ready.
```

Two more factors make things worse: (a) processes are **permanently tied** to their queue and can never move up, and (b) Q3 uses FCFS, so even when Q3 finally runs, a long batch job ahead of P5 can occupy the CPU for a long time (the convoy effect).

3. Optimisation for fairness

Main fix: Aging combined with a Multilevel Feedback Queue (MLFQ).

- Record how long each process has waited. If P5 waits longer than a threshold T (for example 500 ms) in Q3, **promote** it to Q2, and later to Q1 if it still has not been served.
- Once P5 has received service it can be demoted again (or lose its boost), so interactive jobs keep their good response time.
- Equivalently, raise a process's effective priority by one level for every fixed interval it waits, which guarantees that it eventually reaches the top.

Supporting fix: Time-slicing between queues. Divide CPU time among the queues, for example Q1 = 60%, Q2 = 30%, Q3 = 10%. Q3 then has a guaranteed minimum share even when the higher queues are busy.

Further improvement: switch Q3 from FCFS to Round Robin with a larger quantum so that one long batch job cannot hold up P5.

Effect: P5's maximum waiting time becomes bounded (no starvation) while interactive processes still respond quickly. The price is a somewhat more complex scheduler and a slightly smaller CPU share for Q1 and Q2.

Conclusion: starvation arises from strict priority between queues and static queue assignment. Aging (MLFQ promotion) and/or a guaranteed time share for Q3 restores fairness.

Q5. Context-Switch Overhead Debugging

(20 Marks | BL4)

1. Given data

- 20,000 context switches per second
- Average CPU burst per thread = 2 ms
- Very frequent I/O waits
- Preemptive scheduler with a 1 ms quantum

2. Why the switch rate is so high

1. **Quantum shorter than the burst:** with a 1 ms quantum and a 2 ms average burst, each thread is pre-empted at least once per burst (a burst needs about 2 slices). These are **involuntary switches**, and the quantum is much too small, so the scheduler switches more often than it needs to.
2. **Constant I/O blocking:** every time a thread issues I/O it blocks and the CPU must move to another thread (a **voluntary switch**); another switch happens when the I/O completes and the thread is rescheduled or pre-empts the running one.
3. **Too many short-lived threads:** a thread-per-request design creates a large number of runnable and blocked threads, which increases the number of scheduling events.

Each switch costs time: registers must be saved and restored, the kernel is entered and exited, and the scheduler runs, plus **indirect costs** such as cache and TLB pollution. At a typical 5 to 10 microseconds per switch, 20,000 switches per second consume 0.1 to 0.2 s of every second (10 to 20%) without doing useful work. CPU efficiency can be expressed as useful time / (useful time + switch time). With quantum q = 1 ms and switch cost s = 0.01 ms, efficiency = 1 / (1 + 0.01) = 99% in theory, but in this system the cache and TLB penalty and the I/O-driven switches push the real overhead far higher.

3. Two optimisations

Optimisation 1: Increase the time quantum (tune the scheduler).

- Set the quantum slightly above the typical burst (for example 4 to 5 ms, so that about 80% or more of bursts finish inside one quantum).
- Threads then complete their burst or block for I/O before being pre-empted, which eliminates most involuntary switches. An adaptive quantum or MLFQ keeps interactive jobs responsive.
- Trade-off: a very large quantum hurts response time, so choose the value from the burst-time distribution.

Optimisation 2: Cut blocking and thread count (asynchronous I/O + thread pool).

- Replace thread-per-request blocking with an **event-driven, asynchronous I/O model** (epoll, io_uring, non-blocking I/O), so that a small pool of worker threads (about one per core) can serve many requests without blocking.
- Use a fixed **thread pool**, batch I/O requests, and employ **user-level threads or coroutines**, whose switch cost is far lower than a kernel context switch.
- Result: far fewer voluntary switches and much less kernel involvement.

Additional measures: set CPU affinity so a thread stays on one core and keeps its cache warm; use lock-free or fine-grained synchronisation to avoid blocking; and use interrupt coalescing and DMA for I/O.

Conclusion: the high switch rate stems from a quantum (1 ms) that is shorter than the burst (2 ms) and from continual I/O blocking. Raising the quantum and moving to asynchronous I/O with a bounded thread pool reduces the number of switches and gives useful CPU time back.